ClusterSetup

Cisco Ramon

[2026-03-14 Sat 00:35]

Contents

1 Abstract

This documentation presents the complete setup and configuration of a lightweight High Performance Computing (HPC) cluster environment using Rocky Linux and VirtualBox. The guide covers practical implementation of core HPC infrastructure components including shared storage, Slurm workload management, LDAP-based centralized authentication, MPI runtime environment, and centralized software stack management using Spack.

The cluster environment supports multinode job execution, centralized software distribution, distributed MPI applications, and scheduler-aware runtime execution. The setup follows real-world HPC concepts while remaining suitable for learning, experimentation, and practical hands-on training.

2 Preface

This documentation was prepared with the assistance of AI tools for:

- structuring
- explanations
- formatting
- workflow organization
- command documentation
- concept explanation

However, the complete HPC cluster setup, configuration, testing, debugging, and validation were performed manually on a real working cluster environment.

Every major component documented in this guide was configured and tested practically, including:

- cluster networking
- shared storage
- NFS configuration
- Slurm workload manager

- LDAP authentication
- centralized user management
- Spack software stack
- MPI runtime environment
- multinode execution
- scheduler-aware MPI jobs

All commands, workflows, scripts, and execution procedures were tested successfully from start to end.

This guide is designed for:

- students
- beginners
- HPC enthusiasts
- Linux learners
- system administrators
- researchers

who want practical hands-on experience in:

- Linux cluster administration
- HPC infrastructure setup
- distributed systems
- centralized authentication
- workload scheduling
- parallel programming
- MPI execution
- software stack management

The primary goal of this documentation is not only to provide commands, but also to explain the concepts behind real HPC environments in a simple, practical, and beginner-friendly manner.

The setup presented here follows real-world HPC center practices while remaining accessible for learning, experimentation, and self-study.

3 Introduction

High Performance Computing (HPC) clusters are distributed computing systems designed to execute computational workloads across multiple machines simultaneously. Instead of relying on a single system, HPC clusters combine multiple compute nodes to provide higher computational capacity, scalability, and parallel execution capability.

Modern HPC environments are widely used in:

- scientific simulations
- engineering workloads
- computational chemistry
- artificial intelligence
- machine learning
- weather modeling
- large-scale data analysis

A typical HPC cluster contains several important components working together:

Component	Purpose
Login Node	User access and job submission
Compute Nodes	Execute computational workloads
Shared Storage	Shared user data and software
Scheduler	Resource allocation and job management
MPI Runtime	Distributed parallel execution
Authentication System	Centralized user management

This guide focuses on building these components step-by-step in a small virtualized environment using:

- Rocky Linux
- VirtualBox
- Slurm
- OpenMPI

- LDAP
- NFS
- Spack

Although the setup is lightweight and designed for learning purposes, the overall architecture closely follows concepts used in production HPC centers and supercomputers.

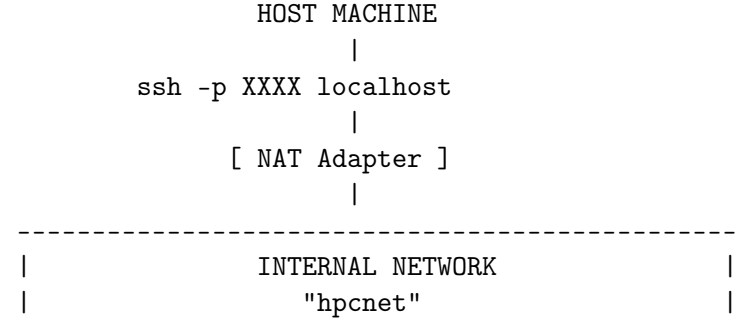
Throughout this documentation, readers will gradually configure:

- cluster networking
- shared storage
- centralized authentication
- workload scheduling
- distributed software stacks
- MPI-based parallel execution
- scheduler-aware job execution

The final environment provides a fully working multinode HPC cluster capable of running distributed parallel applications across multiple compute nodes.

4 Cluster Architecture

The cluster consists of five virtual machines connected using an internal network.



```
-----------------------------------------------

master      10.10.10.1
login       10.10.10.2
compute01   10.10.10.11
compute02   10.10.10.12
compute03   10.10.10.13
```

5 Node Details

5.1 Node Role

1. Master Node

   The master node manages the overall cluster.

   Main responsibilities:

   - Slurm controller
   - NFS server
   - LDAP server
   - Monitoring tools
   - Shared software stack
   - Cluster configuration

2. Login Node

   The login node is the access point for users.

   Typical tasks performed here:

   - SSH login
   - Code compilation
   - Job submission
   - Accessing shared storage

3. Compute Nodes

   Compute nodes run the actual workloads.

   These nodes are mainly used for:

   - MPI jobs

- OpenMP jobs
- Batch workloads
- Parallel execution

5.2 Networking Setup

Two network adapters are used for each VM.

1. NAT Adapter

   Used for:

   - Internet access
   - Package installation
   - SSH access from host system

2. Internal Network

   Used for communication between cluster nodes.

   Network name:

   ```
   hpcnet
   ```

   This network is isolated and only used inside the cluster.

5.3 Static IP Addressing

Each node uses a fixed IP address.

VM Name	Hostname	Internal IP	SSH Port
master	master.local	10.10.10.1	2221
login	login.local	10.10.10.2	2222
compute01	compute01.local	10.10.10.11	2223
compute02	compute02.local	10.10.10.12	2224
compute03	compute03.local	10.10.10.13	2225

Example SSH access:

```
ssh -p 2221 hpcuser@localhost
```

6 Resource Allocation

The following configuration is enough for a lightweight HPC lab. You can increase resources later depending on your host system.

6.1 Master Node

Resource	Value
RAM	2 GB
CPU	2
Disk	30 GB

6.2 Login Node

Resource	Value
RAM	2 GB
CPU	2
Disk	25 GB

6.3 Compute Nodes

Resource	Value
RAM	2 GB
CPU	2
Disk	20 GB

7 Master Node Setup

This section covers the complete setup of the master node. The master node acts as the central management node of the cluster.

Main responsibilities:

- Cluster management
- Shared storage
- Slurm controller
- Monitoring services
- User authentication
- Software management

7.1 Install VirtualBox on Manjaro

I'm using Manjaro (Arch) as my base system.

1. Update packages

   ```
   sudo pacman -Syu
   ```

2. Install VirtualBox and host modules

   ```
   sudo pacman -S virtualbox virtualbox-host-modules-arch
   ```

3. Install VirtualBox extension pack

   ```
   sudo pacman -S virtualbox-ext-oracle
   ```

4. Load VirtualBox kernel module

   ```
   sudo modprobe vboxdrv
   ```

5. Add current user to VirtualBox group

   ```
   sudo usermod -aG vboxusers $USER
   ```

6. Reboot the system

   ```
   sudo reboot
   ```

7.2 Download Rocky Linux

Download Rocky Linux Minimal ISO:
`https://rockylinux.org/download`
Choose:

- x86_64
- Minimal ISO

The minimal installation keeps the environment lightweight and avoids unnecessary packages.

7.3 Create Master VM

Open VirtualBox and create a new virtual machine.

1. VM Name

   ```
   master
   ```

2. Type

   ```
   Linux
   Red Hat (64-bit)
   ```

3. Hardware

Setting	Value
RAM	2046 MB
CPU	2
Disk	30 GB

4. Disk Type

   ```
   VDI
   Dynamically Allocated
   ```

   This configuration is enough for a lightweight HPC lab environment.

7.4 Network Configuration

1. Adapter 1

   ```
   Attached to: NAT
   ```

   Used for:

   - Internet access
   - Package installation
   - SSH access from host machine

2. NAT Port Forwarding Open:

```
Settings -> Network -> Adapter 1 -> Advanced -> Port Forwarding
```

Name	Protocol	Host Port	Guest Port
ssh	TCP	2221	22

3. Adapter 2

   ```
   Attached to: Internal Network
   ```

   Internal Network Name:

   ```
   hpcnet
   ```

   IMPORTANT: All nodes MUST use the same internal network name. This network is used for communication between cluster nodes.

7.5 Rocky Linux Installation

Choose:

```
Minimal Install
```

1. Network and Hostname

   Enable BOTH network adapters.

   Set hostname:

   ```
   master.local
   ```

2. User Creation Create an administrative user during installation.

Field	Value
Username	hpcadmin
Administrator	YES

   After installation, log into the VM.

7.6 Initial System Verification

1. Check Interfaces

   ```
   ip addr
   ```

   Expected:

   - NAT interface should receive:
     - 10.0.2.15
   - Internal interface should exist but may not yet have static IP

7.7 Configure Static Internal Network

1. Check Device Names

   ```
   nmcli device status
   ```

   Expected:

Interface	Purpose
enp0s3	NAT
enp0s8	Internal Network

7.8 Configure Internal Interface

Launch NetworkManager TUI:

```
sudo nmtui
```

Navigate:

```
Edit a connection
```

Select:

```
enp0s8
```

Set:

Field	Value
IPv4 Method	Manual
Address	10.10.10.1/24
Gateway	blank
DNS	blank

DO NOT modify NAT interface.

7.9 Restart Networking

```
sudo systemctl restart NetworkManager
```

7.10 Verify Networking

```
ip addr
```

Expected:

Interface	IP
NAT	10.0.2.x
Internal	10.10.10.1

7.11 SSH Access from Host

From host machine:

```
ssh -p 2221 hpcadmin@localhost
```

If the connection works, port forwarding is configured correctly.

7.12 System Update and Packages

1. Update System

   ```
   sudo dnf update -y
   ```

2. Install Base Packages

   ```
   sudo dnf install -y \
   vim wget curl git \
   net-tools bind-utils \
   htop tmux tree \
   gcc gcc-c++ gcc-gfortran \
   make cmake \
   python3 \
   openssh-server
   ```

   These packages provide:

   - Development tools
   - Network utilities
   - Monitoring utilities
   - Basic administration tools

7.13 Timezone Configuration

Check timezone:

```
timedatectl
```

Set timezone if needed:

```
sudo timedatectl set-timezone Asia/Kolkata
```

7.14 Configure /etc/hosts

Edit:

```
sudo vim /etc/hosts
```

Add:

```
10.10.10.1   master master.local
10.10.10.2   login login.local
10.10.10.11  compute01 compute01.local
10.10.10.12  compute02 compute02.local
10.10.10.13  compute03 compute03.local
```

This allows hostname-based communication between nodes.

7.15 Disable Firewall (Temporary)

For initial setup and testing:

```
sudo systemctl disable --now firewalld
```

NOTE:
Firewall rules can be configured properly later after cluster setup is complete.

7.16 SELinux Configuration

Check status:

```
getenforce
```

Temporarily disable enforcing:

```
sudo setenforce 0
```

Persist configuration:

```
sudo vim /etc/selinux/config
```

Modify:

```
SELINUX=permissive
```

This helps avoid permission-related issues during initial cluster setup.

7.17 Verify Hostname

```
hostnamectl
```

Expected:

```
Static hostname: master.local
```

If hostname is not updated(showing previous), reboot the machine.

At this point, the master node setup is complete and ready for cloning or further cluster configuration.

8 Cloning Nodes

Purpose:

- avoid reinstalling all nodes manually
- simulate real HPC image provisioning workflow

8.1 Create Snapshot

Power off VM:

```
sudo poweroff
```

Create snapshot in VirtualBox:

```
clean-base-master
```

8.2 Clone Nodes

Create FULL CLONES.
IMPORTANT: Enable:

```
Reinitialize the MAC address of all network cards
```

8.3 Clone Names

VM Name
login
compute01
compute02
compute03

8.4 Configure Port Forwarding

1. Login Node

Host Port	Guest Port
2222	22

2. Compute01

Host Port	Guest Port
2223	22

3. Compute02

Host Port	Guest Port
2224	22

4. Compute03

Host Port	Guest Port
2225	22

8.5 Machine ID Regeneration

IMPORTANT: Each cloned Linux machine must have unique machine-id.

8.6 Remove Existing Machine ID

```
sudo rm -f /etc/machine-id
```

8.7 Generate New Machine ID

```
sudo systemd-machine-id-setup
```

8.8 Verify Machine ID

```
cat /etc/machine-id
```

Ensure every node has unique machine-id.

8.9 Configure Hostname

1. Login Node
   Edit hostname:

   ```
   sudo vim /etc/hostname
   ```

   Set:

   ```
   login.local
   ```

   Apply:

   ```
   sudo hostnamectl set-hostname login.local
   ```

2. Compute01
   Set hostname:

   ```
   compute01.local
   ```

   Apply:

   ```
   sudo hostnamectl set-hostname compute01.local
   ```

3. Compute02

```
compute02.local
```

```
sudo hostnamectl set-hostname compute02.local
```

4. Compute03

```
compute03.local
```

```
sudo hostnamectl set-hostname compute03.local
```

8.10 Configure Static Internal IPs

Launch:

```
sudo nmtui
```

Edit internal adapter:

```
enp0s8
```

1. Login Node

```
10.10.10.2/24
```

2. Compute01

```
10.10.10.11/24
```

3. Compute02

```
10.10.10.12/24
```

4. Compute03

```
10.10.10.13/24
```

8.11 Restart Networking

```
sudo systemctl restart NetworkManager
```

8.12 Verify Networking

```
ip addr
```

8.13 Verify Routing

```
ip route
```

Expected:

```
default via 10.0.2.2 dev enp0s3
10.10.10.0/24 dev enp0s8
```

8.14 Verify Connectivity

1. Ping Between Nodes

   ```
   ping master
   ```

   ```
   ping login
   ```

   ```
   ping compute01
   ```

8.15 SSH Verification

From host:

1. Master

   ```
   ssh -p 2221 hpcadmin@localhost
   ```

2. Login

   ```
   ssh -p 2222 hpcadmin@localhost
   ```

3. Compute01

   ```
   ssh -p 2223 hpcadmin@localhost
   ```

4. Compute02

   ```
   ssh -p 2224 hpcadmin@localhost
   ```

5. Compute03

   ```
   ssh -p 2225 hpcadmin@localhost
   ```

9 Shared Infrastructure Services

Goal:

Transform the cluster into a realistic HPC-style shared environment.
This phase includes:

- passwordless SSH
- shared home directories
- NFS server/client configuration
- persistent mounts
- multi-node shared filesystem verification

By the end of this phase:

- all nodes share same /home
- users can access same files from all nodes
- cluster behaves like a real HPC environment

9.1 Cluster Role Architecture

1. Master Node

   Purpose:

   - cluster management
   - NFS server
   - future Slurm controller
   - future Munge authority
   - future LDAP server

   Services:

Service	Purpose
nfs-server	shared storage
future slurmctld	scheduler controller
future munge	authentication

2. Login Node

   Purpose:

   - user login
   - compilation
   - job submission

   Users should:

   - SSH into login node
   - compile programs
   - submit jobs using Slurm

3. Compute Nodes

   Purpose:

   - execute jobs only

   Future services:

Service	Purpose
slurmd	compute daemon
munge	authentication

9.2 Verify User Consistency

IMPORTANT:
NFS shared storage requires same UID/GID across all nodes.
Verify user identity on all nodes:

```
id hpcadmin
```

Expected:

```
uid=1000(hpcadmin) gid=1000(hpcadmin)
```

All nodes must match.

9.3 Passwordless SSH Configuration

Purpose:

- MPI communication
- Slurm execution
- cluster automation
- remote management

SSH trust is generated ONLY from login node.
This mimics real HPC architecture:

```
User -> Login Node -> Compute Nodes
```

9.4 Generate SSH Key on Login Node

SSH into login node from host:

```
ssh -p 2222 hpcadmin@localhost
```

Generate SSH key:

```
ssh-keygen
```

Press Enter for all prompts.
Generated files:

```
~/.ssh/id_rsa
~/.ssh/id_rsa.pub
```

9.5 Copy SSH Keys to Cluster Nodes

1. Master

   ```
   ssh-copy-id hpcadmin@master
   ```

2. Compute01

   ```
   ssh-copy-id hpcadmin@compute01
   ```

3. Compute02

   ```
   ssh-copy-id hpcadmin@compute02
   ```

4. Compute03

   ```
   ssh-copy-id hpcadmin@compute03
   ```

9.6 Verify Passwordless SSH

1. Test Master

   ```
   ssh master hostname
   ```

   Expected:

   ```
   master.local
   ```

2. Test Compute01

   ```
   ssh compute01 hostname
   ```

   Expected:

   ```
   compute01.local
   ```

3. Test Compute02

   ```
   ssh compute02 hostname
   ```

4. Test Compute03

   ```
   ssh compute03 hostname
   ```

   No password prompts should appear.

9.7 HPC Concept — Why Passwordless SSH Matters

Passwordless SSH becomes essential for:

Feature	Purpose
MPI	remote process launch
Slurm	distributed execution
automation	orchestration
monitoring	remote administration

9.8 NFS Shared Storage Setup

Purpose:

Provide shared home directories across all cluster nodes.
This is a core HPC architecture requirement.
Without shared storage:

- source code differs per node
- binaries differ
- scripts unavailable across nodes
- MPI execution becomes difficult

Real HPC systems commonly use:

- NFS
- Lustre
- BeeGFS
- GPFS

This lab begins with NFS.

9.9 Install NFS Packages

Install on ALL nodes:

```
sudo dnf install -y nfs-utils
```

9.10 Configure NFS Server on Master

SSH into master node.

9.11 Enable NFS Services

```
sudo systemctl enable --now nfs-server
```

Verify:

```
systemctl status nfs-server
```

Expected:

```
active (running)
```

9.12 Configure NFS Exports

Edit exports file:

```
sudo vim /etc/exports
```

Add:

```
/home 10.10.10.0/24(rw,sync,no_root_squash)
```

9.13 Export Option Explanation

Option	Meaning
rw	read/write access
sync	synchronous writes
no_root_squash	preserve root privileges

9.14 Apply Exports

```
sudo exportfs -rav
```

9.15 Verify Exports

```
sudo exportfs -v
```

Expected export:

```
/home 10.10.10.0/24(...)
```

9.16 Configure NFS Clients

Perform on:

- login
- compute01
- compute02
- compute03

9.17 Backup Existing Home Directory

```
sudo mv /home /home.local
```

Purpose:
Preserve original local home directory.

9.18 Create New Mount Point

```
sudo mkdir /home
```

9.19 Mount Shared NFS Home

```
sudo mount master:/home /home
```

9.20 Verify Mount

```
df -h
```

Expected:

```
master:/home
```

mounted on:

```
/home
```

9.21 Verify Shared Filesystem

From login node:

```
touch ~/shared_test
```

From compute01:

```
ls ~
```

Expected:

```
shared_test
```

This confirms successful shared filesystem operation.

9.22 Configure Persistent NFS Mounts

Without this configuration:

- NFS mount disappears after reboot

9.23 Configure /etc/fstab

On ALL client nodes:

```
sudo vim /etc/fstab
```

Add:

```
master:/home   /home   nfs   defaults   0 0
```

9.24 Verify fstab Before Reboot

IMPORTANT:
Always validate fstab before rebooting.
Run:

```
sudo mount -a
```

If no errors appear:

- fstab configuration is correct

9.25 Reboot Validation Procedure

Purpose:

Validate persistent cluster infrastructure after reboot.

This is important real-world HPC administration practice.

9.26 Reboot Order

1. Reboot Master First

   ```
   sudo reboot
   ```

   Wait until:

   - SSH becomes available
   - NFS server active

   Verify:

   ```
   systemctl status nfs-server
   ```

2. Reboot Login Node

   ```
   sudo reboot
   ```

3. Reboot Compute Nodes

   ```
   sudo reboot
   ```

   Perform for:

   - compute01
   - compute02
   - compute03

9.27 Verify Persistent Mounts After Reboot

On login node:

```
df -h
```

Expected:

```
master:/home
```

mounted automatically.

9.28 Verify Active NFS Mount

```
mount | grep home
```

Expected:

```
master:/home on /home type nfs
```

9.29 Verify Shared Files After Reboot

```
ls ~
```

Expected:

- previously created shared files still visible

10 Munge Authentication + Slurm Scheduler

Goal:

Transform the cluster into a scheduler-managed HPC environment.
This phase includes:

- Munge authentication
- Slurm controller setup
- Compute node daemon setup
- Scheduler configuration
- Partition creation
- Multi-node execution
- Resource allocation testing

By the end of this phase:

- cluster nodes authenticate securely
- Slurm manages compute resources
- jobs execute across nodes
- scheduler topology becomes operational

10.1 HPC Scheduler Architecture

1. Master Node

   Purpose:

   - Slurm controller
   - cluster scheduler
   - resource manager
   - Munge authority

   Services:

Service	Purpose
munged	cluster authentication
slurmctld	scheduler controller

2. Login Node

   Purpose:

   - user login
   - compilation
   - job submission

   Users should:

   - login here
   - compile here
   - submit jobs here

3. Compute Nodes

   Purpose:

   - execute jobs

   Services:

Service	Purpose
munged	authentication
slurmd	compute execution daemon

10.2 IMPORTANT Rocky Linux 10 Issue

Originally the cluster was created using:

```
Rocky Linux 10
```

Problem encountered:

```
No match for argument: slurm
```

Reason:

- Rocky Linux 10 HPC ecosystem still immature
- Slurm packages unavailable in standard repos
- OpenHPC EL10 support incomplete

Resolution:

- cluster rebuilt using Rocky Linux 9 Minimal

Important HPC lesson:
Production HPC clusters usually avoid bleeding-edge enterprise releases because:

- scheduler ecosystems lag behind
- HPC repos may not exist
- compatibility becomes unstable

10.3 Install EPEL Repository

Install on ALL nodes:

```
sudo dnf install -y epel-release
```

10.4 Enable CRB Repository

Required on Rocky Linux 9.
Install on ALL nodes:

```
sudo crb enable
```

10.5 Install Munge + Slurm Packages

Install on ALL nodes:

```
sudo dnf install -y \
munge munge-libs \
slurm slurm-slurmd slurm-slurmctld
```

10.6 Verify Installed Packages

1. Verify Munge

   ```
   rpm -qa | grep munge
   ```

   Expected:

   ```
   munge
   munge-libs
   ```

2. Verify Slurm

   ```
   rpm -qa | grep slurm
   ```

   Expected:

   ```
   slurm
   slurm-slurmd
   slurm-slurmctld
   ```

10.7 Verify System Users

1. Verify Munge User

   ```
   id munge
   ```

2. Verify Slurm User

   ```
   id slurm
   ```

10.8 IMPORTANT Issue Encountered — Missing slurm User

Problem:

```
id: 'slurm': no such user
```

Reason:

Some Slurm packages on Rocky Linux do not automatically create the slurm user.

Resolution:

Create manually on ALL nodes.

10.9 Create Slurm Group

```
sudo groupadd slurm
```

10.10 Create Slurm User

```
sudo useradd -r -g slurm -s /sbin/nologin slurm
```

Explanation:

Option	Meaning
-r	system account
-g slurm	primary group
-s /sbin/nologin	disable shell login

10.11 Verify Slurm User

```
id slurm
```

Expected:

```
uid=...
gid=...
```

10.12 IMPORTANT HPC Security Concept

Cluster daemons should not run permanently as root.

Therefore:

- Munge runs as munge user
- Slurm runs as slurm user

This is standard enterprise daemon security practice.

10.13 Configure Munge Authentication

Purpose:

Create secure trust relationship between all cluster nodes.

IMPORTANT:

All nodes MUST use the same:

```
/etc/munge/munge.key
```

This key acts as the cluster-wide shared secret.

10.14 Generate Munge Key on Master

ONLY on master node:

```
sudo /usr/sbin/create-munge-key
```

10.15 Verify Munge Key

```
ls -l /etc/munge/munge.key
```

Expected:

```
-rw-------
```

owned by:

```
munge munge
```

10.16 IMPORTANT Mistake Encountered — Munge Key Generated on All Nodes

Problem:

Munge key accidentally generated independently on all nodes.

Result:

Nodes could not authenticate each other.

Important HPC lesson:

Munge works using a:

```
shared trust model
```

All nodes MUST share identical munge.key.

10.17 Correct Munge Key Distribution

1. Copy Key Temporarily

   On master:

   ```
   sudo cp /etc/munge/munge.key /tmp/
   ```

2. Temporary Permission Adjustment

   ```
   sudo chmod 644 /tmp/munge.key
   ```

   Purpose:

   Allow normal user SCP transfer.

10.18 IMPORTANT SSH Root Login Issue

Problem encountered:

```
Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password)
```

Reason:
Using:

```
sudo scp ...
```

attempted root SSH login.
Rocky Linux disables direct root SSH login by default.
Resolution:
Use normal user SCP.

10.19 Copy Munge Key to Login Node

```
scp /tmp/munge.key login:/tmp/
```

10.20 Copy Munge Key to Compute01

```
scp /tmp/munge.key compute01:/tmp/
```

10.21 Copy Munge Key to Compute02

```
scp /tmp/munge.key compute02:/tmp/
```

10.22 Copy Munge Key to Compute03

```
scp /tmp/munge.key compute03:/tmp/
```

10.23 Remove Temporary Key Copy

On master:

```
sudo rm -f /tmp/munge.key
```

10.24 Install Munge Key on Client Nodes

Perform on:

- login
- compute01
- compute02
- compute03

10.25 Stop Munge Service

```
sudo systemctl stop munge
```

10.26 Install Key

```
sudo mv /tmp/munge.key /etc/munge/
```

10.27 Fix Ownership

```
sudo chown munge:munge /etc/munge/munge.key
```

10.28 Fix Permissions

```
sudo chmod 400 /etc/munge/munge.key
```

10.29 Start Munge

On ALL nodes:

```
sudo systemctl enable --now munge
```

10.30 IMPORTANT Issue Encountered — Munge Not Running on Master

Problem:

```
munge: Error: Failed to access "/var/run/munge/munge.socket.2"
```

Reason:
Munge service never started on master.
Resolution:

```
sudo systemctl enable --now munge
```

Important lesson:
Difference between:

State	Meaning
inactive	service not started
failed	service crashed

10.31 Verify Munge Service

```
systemctl status munge
```

Expected:

```
active (running)
```

10.32 Verify Munge Authentication

From master:

```
munge -n | ssh compute01 unmunge
```

10.33 Test Compute02

```
munge -n | ssh compute02 unmunge
```

10.34 Test Compute03

```
munge -n | ssh compute03 unmunge
```

10.35 Test Login Node

```
munge -n | ssh login unmunge
```

Expected:

```
STATUS: Success
```

10.36 HPC Concepts Learned

Munge introduced:

- cluster-wide authentication
- distributed trust model
- shared secret authentication
- daemon security
- inter-node trust validation

10.37 Configure Slurm Scheduler

Purpose:
Enable centralized resource scheduling.
IMPORTANT:

```
/etc/slurm/slurm.conf
```

must be IDENTICAL on ALL nodes.

10.38 IMPORTANT Issue Encountered — Default localhost Configuration

Problem:

```
sinfo
-> localhost
```

Reason:
Default example slurm.conf remained active.
Resolution:
Replace configuration with custom cluster topology.
Important HPC lesson:
Incorrect topology configuration is one of the most common Slurm admin problems.

10.39 Create Slurm Configuration

On master:

```
sudo vim /etc/slurm/slurm.conf
```

Replace ENTIRE file with:

```
ClusterName=hpccluster
SlurmctldHost=master

MpiDefault=none
ProctrackType=proctrack/cgroup
ReturnToService=2

SlurmctldPidFile=/run/slurmctld.pid
SlurmdPidFile=/run/slurmd.pid

SlurmdSpoolDir=/var/spool/slurmd
StateSaveLocation=/var/spool/slurmctld

SlurmUser=slurm

SwitchType=switch/none
TaskPlugin=task/affinity

SchedulerType=sched/backfill

SelectType=select/cons_tres
SelectTypeParameters=CR_Core

AuthType=auth/munge

NodeName=compute01 CPUs=2 State=UNKNOWN
NodeName=compute02 CPUs=2 State=UNKNOWN
NodeName=compute03 CPUs=2 State=UNKNOWN

PartitionName=debug Nodes=ALL Default=YES MaxTime=INFINITE State=UP
```

10.40 Configuration Explanation

Parameter	Purpose
ClusterName	cluster identifier
SlurmctldHost	controller node
NodeName	compute nodes
CPUs=2	available CPUs
PartitionName	scheduler queue

10.41 Create Slurm Runtime Directories

1. On Master

```
sudo mkdir -p /var/spool/slurmctld
```

```
sudo mkdir -p /var/spool/slurmd
```

10.42 Set Ownership

```
sudo chown slurm:slurm /var/spool/slurmctld
```

```
sudo chown slurm:slurm /var/spool/slurmd
```

10.43 Create slurmd Directories on Compute Nodes

1. Compute01

```
ssh compute01 sudo -S mkdir -p /var/spool/slurmd
```

2. Compute02

```
ssh compute02 sudo -S mkdir -p /var/spool/slurmd
```

3. Compute03

```
ssh compute03 sudo -S mkdir -p /var/spool/slurmd
```

10.44 IMPORTANT sudo Over SSH Issue

Problem:

```
sudo: a terminal is required to read the password
```

Resolution:
Use:

```
sudo -S
```

Important Linux administration concept:
sudo over non-interactive SSH requires stdin password handling.

10.45 Fix Ownership on Compute Nodes

1. Compute01

   ```
   ssh compute01 sudo -S chown slurm:slurm /var/spool/slurmd
   ```

2. Compute02

   ```
   ssh compute02 sudo -S chown slurm:slurm /var/spool/slurmd
   ```

3. Compute03

   ```
   ssh compute03 sudo -S chown slurm:slurm /var/spool/slurmd
   ```

10.46 Verify Runtime Directories

```
ls -ld /var/spool/slurmctld
```

Expected:

```
slurm slurm
```

10.47 Distribute slurm.conf

1. Login

   ```
   scp /etc/slurm/slurm.conf login:/tmp/
   ```

2. Compute01

   ```
   scp /etc/slurm/slurm.conf compute01:/tmp/
   ```

3. Compute02

   ```
   scp /etc/slurm/slurm.conf compute02:/tmp/
   ```

4. Compute03

   ```
   scp /etc/slurm/slurm.conf compute03:/tmp/
   ```

10.48 Install Configuration on Nodes

On ALL nodes:

```
sudo mv /tmp/slurm.conf /etc/slurm/slurm.conf
```

10.49 Start Slurm Controller

ONLY on master:

```
sudo systemctl enable --now slurmctld
```

10.50 Verify Controller

```
systemctl status slurmctld
```

Expected:

```
active (running)
```

10.51 Start slurmd on Compute Nodes

On:

- compute01
- compute02
- compute03

```
sudo systemctl enable --now slurmd
```

10.52 Verify slurmd

```
systemctl status slurmd
```

Expected:

```
active (running)
```

10.53 Reconfigure Slurm

On master:

```
sudo scontrol reconfigure
```

10.54 IMPORTANT Issue Encountered — Nodes in DRAIN State

Problem:
Nodes entered:

```
DRAIN
```

after CPU topology changes.
Reason:
Slurm detected mismatch between:

- configured CPUs
- detected hardware topology

Resolution:
Clear stale slurmd state.

10.55 Stop slurmd

On compute nodes:

```
sudo systemctl stop slurmd
```

10.56 Remove Old State

```
sudo rm -rf /var/spool/slurmd/*
```

10.57 Restart slurmd

```
sudo systemctl start slurmd
```

10.58 Resume Nodes

On master:

```
sudo scontrol update NodeName=compute01 State=RESUME
sudo scontrol update NodeName=compute02 State=RESUME
sudo scontrol update NodeName=compute03 State=RESUME
```

10.59 Verify Cluster Status

```
sinfo
```

Expected:

```
PARTITION AVAIL TIMELIMIT NODES STATE NODELIST
debug* up infinite 3 idle compute[01-03]
```

10.60 Verify Node Details

```
scontrol show nodes
```

This displays:

- CPU information
- memory
- node states
- scheduler topology
- resource allocation

10.61 Test Slurm Execution

1. Single Node Test

   From login node:

   ```
   srun hostname
   ```

   Expected:

   ```
   compute01.local
   ```

10.62 Multi-node Execution

```
srun -N 3 hostname
```

Expected:

```
compute01.local
compute02.local
compute03.local
```

10.63 Multi-task Multi-node Execution

```
srun -N 3 -n 6 hostname
```

This verifies:

- task distribution
- multi-node scheduling
- CPU allocation

10.64 Multi-core Single-node Execution

```
srun -N 1 -n 2 hostname
```

Expected:

```
compute01.local
compute01.local
```

10.65 IMPORTANT Slurm Scheduling Concept

Earlier:

```
srun -N 1 -n 2 hostname
```

failed because nodes initially had:

```
CPUs=1
```

After increasing VM CPUs and updating slurm.conf:

```
CPUs=2
```

the scheduler successfully allocated 2 tasks on a single node.
Important HPC lesson:
Slurm strictly enforces:

- CPU limits
- task allocation
- topology constraints
- resource consistency

11 Shared Software Filesystem (/apps)

Goal:
Create a dedicated shared software filesystem for:

- Spack
- compilers
- MPI stacks
- scientific applications
- modulefiles
- HPC software hierarchy

This phase transitions the cluster from:

```
basic scheduler infrastructure
```

to:

```
realistic HPC software environment
```

11.1 Why Separate /apps Filesystem?

Initially:

- each VM had a 30 GB local disk
- /home was exported from master over NFS

Concern encountered:

```
Shared /home only had ~30 GB available.
Spack and HPC software stacks can become very large.
```

Important HPC storage concept:
Real HPC clusters usually separate:

Filesystem	Purpose
/home	user files
/apps	software stack
/scratch	temporary runtime data
/archive	long-term storage

Therefore a dedicated:

```
/apps
```

filesystem was created.
This is significantly closer to real HPC architecture.

11.2 IMPORTANT Storage Architecture Concept

Each VM still has its own independent local storage.
Example:

Node	Local Disk
master	30 GB
login	30 GB
compute01	30 GB
compute02	30 GB
compute03	30 GB

When:

```
master:/home
```

was mounted via NFS:

- compute node local /home became hidden
- but local disks still exist

Only the exported filesystem becomes shared.
Therefore:

```
/apps
```

was created as a dedicated shared software layer.

11.3 Add New Virtual Disk to Master

A second virtual disk was attached ONLY to:

```
master
```

Purpose:

- centralized software storage
- shared application filesystem
- Spack software stack

Disk size selected:

```
60 GB
```

Disk type:

```
VDI (Dynamically Allocated)
```

11.4 Verify New Disk Detection

After booting master:

```
lsblk
```

Expected:

```
sda    30G
sdb    60G
```

Important:

Device	Purpose
sda	OS disk
sdb	New apps disk

Only:

```
sdb
```

was modified.

11.5 Create GPT Partition Table

```
sudo fdisk /dev/sdb
```

Inside fdisk:
Create GPT label:

```
g
```

Create partition:

```
n
```

Accept defaults.
Write changes:

```
w
```

11.6 Verify Partition

```
lsblk
```

Expected:

```
sdb
 sdb1
```

11.7 Create XFS Filesystem

```
sudo mkfs.xfs /dev/sdb1
```

Why XFS?

- default enterprise filesystem
- highly scalable
- excellent for large filesystems
- commonly used in HPC

11.8 Create Mount Point

```
sudo mkdir -p /apps
```

11.9 Get Filesystem UUID

```
sudo blkid /dev/sdb1
```

Example:

```
UUID="xxxxxxxx-xxxx-xxxx"
```

11.10 Configure Persistent Mount

Edit:

```
sudo vim /etc/fstab
```

Add:

```
UUID=YOUR_UUID_HERE   /apps   xfs   defaults   0 0
```

Important Linux administration concept:
UUIDs are preferred over:

```
/dev/sdb1
```

because device names can change after reboot.

11.11 Mount Filesystem

```
sudo mount -a
```

11.12 Verify Mount

```
df -h
```

Expected:

```
/apps
```

mounted with approximately:

```
60 GB
```

11.13 Configure Ownership

```
sudo chown hpcadmin:hpcadmin /apps
```

11.14 Create Shared HPC Software Hierarchy

```
mkdir -p /apps/{spack,modulefiles,software,sources}
```

Resulting structure:

```
/apps
 spack
 modulefiles
 software
 sources
```

11.15 HPC Software Hierarchy Concept

Purpose of directories:

Directory	Purpose
spack	Spack package manager
modulefiles	custom modulefiles
software	installed software
sources	downloaded sources

This is similar to production HPC software layouts.

11.16 Export /apps via NFS

Edit exports file:

```
sudo vim /etc/exports
```

Add:

```
/apps 10.10.10.0/24(rw,sync,no_root_squash)
```

11.17 Apply NFS Exports

```
sudo exportfs -rav
```

11.18 Verify Exports

```
showmount -e localhost
```

Expected:

```
/apps
/home
```

11.19 Create Mount Points on Client Nodes

On:

- login
- compute01
- compute02

- compute03

Create:

```
sudo mkdir -p /apps
```

11.20 Configure Persistent NFS Mounts

Edit:

```
sudo vim /etc/fstab
```

Add:

```
master:/apps   /apps   nfs   defaults,_netdev   0 0
```

11.21 Mount Shared /apps Filesystem

```
sudo mount -a
```

11.22 Verify Shared Mount

```
df -h
```

Expected:

```
master:/apps
```

mounted on all nodes.

11.23 Verify Shared Filesystem Functionality

On master:

```
touch /apps/testfile
```

On compute node:

```
ls /apps
```

Expected:

```
testfile
```

This verifies:

- shared software filesystem
- NFS functionality
- cluster-wide software visibility

11.24 HPC Architecture After /apps Setup

Current cluster storage architecture:

Filesystem	Purpose
/home	user files
/apps	software stack
local disks	local runtime storage

This resembles real HPC environments significantly more closely.

12 Spack Software Stack + MPI Runtime Validation

Goal:

Create centralized HPC software stack using:

- Spack
- shared software filesystem
- shared compilers
- shared MPI stack
- scheduler-aware runtime execution

This phase validates:

- software distribution
- multinode execution
- Slurm integration
- MPI runtime functionality

After completing this section you will be able to:

- install HPC software using Spack
- manage multiple compiler versions
- manage multiple MPI versions

- load software dynamically
- compile MPI applications
- execute distributed jobs using Slurm
- monitor and debug jobs
- understand performance metrics

12.1 IMPORTANT HPC Software Stack Concept

Instead of:

```
dnf install openmpi
```

a centralized HPC software stack was created.
Benefits:

- software installed once
- accessible cluster-wide
- compiler hierarchy support
- reproducible builds
- module-based software environments
- centralized software management

This approach is significantly closer to real HPC centers.
In production HPC environments:

- operating system packages are separated
- scientific software stack is separated
- compiler stacks are separated
- MPI stacks are separated

This separation improves:

- portability
- reproducibility
- scalability
- maintenance

12.2 IMPORTANT Shared Filesystem Concept

The cluster uses:

`/apps`

as shared application storage.
Because:

`/apps`

is mounted cluster-wide through NFS:

- software installed once becomes visible everywhere
- compute nodes automatically access installed software
- users share common software stack
- recompilation is unnecessary on compute nodes

This is one of the MOST important HPC architecture concepts.

12.3 Install Development Tools

Installed manually on ALL nodes:

```
sudo dnf groupinstall -y "Development Tools"
```

Purpose:
Provides:

- gcc
- g++
- gfortran
- make
- linker tools
- build utilities
- development environment

Important note:

```
System compilers are still required even when using Spack.
```

Reason:
Spack builds software from source code and therefore still requires:

- assembler
- linker
- basic compiler toolchain

12.4 Move into Shared Application Filesystem

Run on:

```
master
```

Move into shared applications directory:

```
cd /apps
```

Verify:

```
pwd
```

Expected:

```
/apps
```

12.5 Clone Spack into Shared Filesystem

Clone Spack:

```
git clone https://github.com/spack/spack.git
```

Expected:

```
/apps/spack
```

This creates centralized package manager installation.

12.6 Enter Spack Directory

```
cd spack
```

Verify:

```
pwd
```

Expected:

```
/apps/spack
```

12.7 Initialize Spack Environment

Initialize Spack:

```
source share/spack/setup-env.sh
```

Purpose:
Adds:

- spack command
- shell integration
- environment variables
- package management environment

to current shell session.
Without initialization:

- spack command will not work
- spack load will fail
- software environments will not activate

12.8 IMPORTANT Shell Environment Concept

`source`

does NOT execute script normally.
Instead:

`it modifies current shell environment`

This is extremely important in:

- Spack
- modules
- MPI environments
- compiler environments

12.9 Make Spack Permanent

To avoid sourcing manually every session:
Edit:

```
vim ~/.bashrc
```

Add:

```
source /apps/spack/share/spack/setup-env.sh
```

Reload shell:

```
source ~/.bashrc
```

Verify:

```
spack --version
```

Expected:

- Spack version information

12.10 Clone Additional Spack Repository

Clone repository:

```
git clone https://github.com/spack/spack-packages.git
```

Purpose:
Adds additional package definitions.

12.11 Configure Spack Repository

```
spack repo set --destination "$(pwd)/spack-packages" builtin
```

Purpose:
Configure additional repository support.
This allows:

- package metadata
- package recipes
- software definitions

to become visible to Spack.

12.12 Verify Spack Functionality

```
spack info gcc
```

Expected:
Detailed GCC package information.
This verifies:

- Spack environment working
- package repository functional
- package metadata accessible

12.13 IMPORTANT HPC Software Stack Concept

Spack operates as:

`source-based HPC package manager`

Unlike:

Package Manager	Purpose
dnf	operating system
Spack	HPC software ecosystem

Production HPC systems commonly separate:

- OS packages
- scientific software stacks

exactly like this setup.

12.14 IMPORTANT Spack Build Concept

When installing software using Spack:

`software is compiled from source`

Advantages:

- architecture optimization
- compiler flexibility
- dependency control
- reproducibility
- variant support

Disadvantages:

- longer installation time

12.15 Detect Available Compilers

Before installing software:

```
spack compiler find
```

Purpose:
Detect system compilers available for building software.
Verify detected compilers:

```
spack compilers
```

Expected:

- compiler list

12.16 Install GCC Using Spack

Install GCC version:

```
spack install gcc@13.4.0
```

Purpose:
Create dedicated HPC compiler stack.
This may take considerable time because:

- GCC itself is compiled from source
- dependencies are built
- optimization occurs

12.17 IMPORTANT Multiple Compiler Version Concept

Production HPC systems frequently maintain multiple compiler versions simultaneously.
Example:

```
gcc@11
gcc@12
gcc@13
intel
nvhpc
clang
```

Reasons:

- application compatibility
- compiler ABI differences
- reproducibility
- benchmark consistency

12.18 Verify Installed Software

List installed packages:

```
spack find
```

Expected example:

```
==> 1 installed package
gcc@13.4.0
```

12.19 Load Installed GCC Compiler

```
spack load gcc@13.4.0
```

Purpose:
Activate compiler environment.
Verify compiler path:

```
which gcc
```

Expected:

- Spack GCC path

Verify version:

```
gcc --version
```

Expected:

- GCC 13.4.0

12.20 IMPORTANT Runtime Environment Concept

When software is loaded:

- PATH changes
- LD_LIBRARY_PATH changes
- MANPATH changes

This allows applications to locate:

- executables
- shared libraries
- runtime dependencies

automatically.

12.21 Register Newly Installed Compiler

After installing GCC:

```
spack compiler find
```

Purpose:
Allow Spack to use newly installed GCC for future package builds.
Verify:

```
spack compilers
```

Expected:

- new GCC compiler entry

12.22 Install OpenMPI Using Spack

Install MPI runtime:

```
spack install openmpi
```

Purpose:
Create shared MPI runtime stack.
This installation includes:

- MPI runtime
- MPI compiler wrappers
- MPI libraries
- distributed runtime environment

12.23 IMPORTANT MPI Compiler Dependency Concept

MPI stacks are tightly coupled with compiler toolchains.
Example:

```
openmpi built using gcc@13
```

This is important because:

- ABI compatibility matters
- runtime compatibility matters
- mixed compiler environments may fail

12.24 Verify Installed Software Tree

Show dependency tree:

```
spack find -d
```

Purpose:
Displays:

- dependency hierarchy
- compiler hierarchy
- runtime dependencies

Example:

```
openmpi
   ^gcc@13.4.0
```

12.25 IMPORTANT Spack Spec Concept

Every Spack installation contains:

- package version
- compiler information
- dependency tree
- unique installation hash

Example:

```
openmpi@5.0.7%gcc@13.4.0
```

Meaning:

Component	Meaning
openmpi	package
5.0.7	package version
gcc@13.4.0	compiler used

12.26 IMPORTANT Hash-Based Software Loading

Sometimes multiple builds of same version exist because of:

- different compilers
- different variants
- different dependencies

Spack assigns:

```
unique hashes
```

View hashes:

```
spack find -l
```

Expected:

```
abc1234 gcc@13.4.0
xyz5678 openmpi@5.0.7
```

Load using hash:

```
spack load /xyz5678
```

This is extremely common in production HPC systems.

12.27 Load OpenMPI Environment

```
spack load openmpi
```

Verify:

```
which mpicc
```

Expected:

- Spack OpenMPI wrapper path

Verify MPI runtime:

```
mpirun --version
```

Expected:

- OpenMPI version information

12.28 IMPORTANT MPI Wrapper Concept

```
mpicc
```

is NOT a compiler itself.
It is a wrapper that automatically injects:

- MPI headers
- MPI libraries
- runtime linker flags

around GCC.
This is a critical HPC build concept.

12.29 Verify Loaded Software

Display loaded packages:

```
spack find --loaded
```

Unload OpenMPI:

```
spack unload openmpi
```

Unload all software:

```
spack unload --all
```

Verify:

```
spack find --loaded
```

Expected:

- no loaded packages

12.30 Create MPI Performance Validation Program

Create source file:

```
vim mpi_sum.c
```

Purpose:

- validate MPI runtime
- validate multinode execution
- validate scheduler-aware execution
- compare serial vs parallel runtime
- demonstrate reduction operations
- calculate speedup
- calculate efficiency

Paste:

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000000

double serial_sum()
{
```

```
    double sum = 0.0;

    for(long long int i = 0; i < N; i++)
    {
        sum += i;
    }

    return sum;
}

int main(int argc, char *argv[])
{
    int rank, size;

    long long int start_index;
    long long int end_index;
    long long int chunk;

    double local_sum = 0.0;
    double global_sum = 0.0;

    double serial_start;
    double serial_end;

    double parallel_start;
    double parallel_end;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if(rank == 0)
    {
        serial_start = MPI_Wtime();

        double serial_result = serial_sum();

        serial_end = MPI_Wtime();
```

```
    printf("\n");
    printf("========== SERIAL COMPUTATION ==========\n");
    printf("Sum = %.0f\n", serial_result);
    printf("Time taken = %f seconds\n",
           serial_end - serial_start);
}

MPI_Barrier(MPI_COMM_WORLD);

chunk = N / size;

start_index = rank * chunk;

if(rank == size - 1)
    end_index = N;
else
    end_index = start_index + chunk;

parallel_start = MPI_Wtime();

for(long long int i = start_index;
    i < end_index;
    i++)
{
    local_sum += i;
}

MPI_Reduce(&local_sum,
           &global_sum,
           1,
           MPI_DOUBLE,
           MPI_SUM,
           0,
           MPI_COMM_WORLD);

parallel_end = MPI_Wtime();

if(rank == 0)
{
    double parallel_time =
```

```
            parallel_end - parallel_start;

        double serial_time =
            serial_end - serial_start;

        printf("\n");
        printf("========== PARALLEL COMPUTATION ==========\n");
        printf("Processes used = %d\n", size);
        printf("Sum = %.0f\n", global_sum);
        printf("Time taken = %f seconds\n",
               parallel_time);

        printf("\n");
        printf("========== PERFORMANCE ==========\n");
        printf("Speedup = %f\n",
               serial_time / parallel_time);

        printf("Efficiency = %f\n",
               (serial_time / parallel_time) / size);
    }

    MPI_Finalize();

    return 0;
}
```

12.31 IMPORTANT MPI Reduction Concept

This program uses:

```
MPI_Reduce()
```

Purpose:
Combine partial sums from all MPI ranks into:

```
single global result
```

Flow:

```
local sums
    ↓
```

```
MPI_Reduce
    ↓
global sum
```

This is one of the MOST important MPI collective communication operations.

12.32 IMPORTANT Parallel Execution Concept

Serial execution:

```
single process computes entire workload
```

Parallel execution:

```
multiple MPI ranks divide workload
```

Example:

Rank	Work
rank0	0 –> 25%
rank1	25% –> 50%
rank2	50% –> 75%
rank3	75% –> 100%

Advantages:

- reduced execution time
- distributed workload
- scalable computation

12.33 Load Software Before Compilation

Initialize Spack:

```
source /apps/spack/share/spack/setup-env.sh
```

Load compiler:

```
spack load gcc@13.4.0
```

Load MPI runtime:

```
spack load openmpi
```

Verify:

```
which mpicc
```

Expected:

- MPI compiler wrapper path

12.34 Compile MPI Program

Compile:

```
mpicc mpi_sum.c -o mpi_sum
```

Verify executable:

```
ls
```

Expected:

```
mpi_sum
```

12.35 IMPORTANT MPI Compilation Concept

MPI programs are compiled using:

```
mpicc
```

instead of:

```
gcc
```

because MPI wrapper automatically injects:

- MPI headers
- MPI libraries
- linker flags

12.36 Interactive MPI Runtime Validation

Run interactively:

```
mpirun -np 4 ./mpi_sum
```

Expected:

```
========== SERIAL COMPUTATION ==========
...

========== PARALLEL COMPUTATION ==========
...

========== PERFORMANCE ==========
...
```

This validates:

- MPI runtime
- MPI communication
- reduction operations
- distributed execution

12.37 IMPORTANT Shared Filesystem Advantage

Because:

- source code
- executable
- software stack

exist on shared filesystem:

```
/apps
or
/home
```

compute nodes automatically access:

- compiled binaries
- runtime libraries
- MPI stack

without recompilation.
This is one of the major HPC architecture advantages.

12.38 Create Slurm Batch Script

Create batch script:

```
vim submit.sh
```

Paste:

```
#!/bin/bash
#SBATCH -N 3
#SBATCH --ntasks-per-node=1
#SBATCH --job-name=test
#SBATCH --output=%J.out
#SBATCH --error=%J.err
#SBATCH --time=01:00:00
##SBATCH --nodelist=compute03,compute01

source /apps/spack/share/spack/setup-env.sh

spack load openmpi

mpirun -np $SLURM_NTASKS ./$1
```

Make executable:

```
chmod +x submit.sh
```

12.39 IMPORTANT Slurm Script Explanation

Line	Purpose
#SBATCH -N 3	request 3 nodes
–ntasks-per-node=1	one MPI rank per node
–job-name=test	job name
–output=%J.out	stdout file
–error=%J.err	stderr file
–time=01:00:00	walltime limit

12.40 IMPORTANT Runtime Environment Section

Inside batch script:

`source /apps/spack/share/spack/setup-env.sh`

initializes Spack environment.

`spack load openmpi`

loads MPI runtime stack.
Without these:

- mpirun may fail
- MPI libraries may not load
- runtime linker errors may occur

12.41 IMPORTANT Slurm Runtime Variable

`$SLURM_NTASKS`

contains:

`total MPI process count allocated by Slurm`

Example:

Nodes	Tasks per node	Total
3	1	3

Therefore:

`mpirun -np $SLURM_NTASKS`

automatically launches correct number of MPI ranks.

12.42 Submit MPI Job

Submit job:

```
sbatch submit.sh mpi_sum
```

Expected:

```
Submitted batch job 101
```

This validates:

- Slurm scheduler
- distributed allocation
- multinode MPI execution
- scheduler-aware runtime launch

12.43 Monitor Jobs

Check user jobs:

```
squeue --me
```

Expected:

```
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
```

12.44 IMPORTANT Job States

State	Meaning
PD	pending
R	running
CG	completing
CD	completed
F	failed

12.45 View Detailed Job Information

Show detailed job info:

```
scontrol show job <jobid>
```

Example:

```
scontrol show job 101
```

Displays:

- allocated nodes
- CPUs
- runtime
- scheduling information
- execution directory
- output locations

12.46 Check Cluster Status

Check cluster nodes:

```
sinfo
```

Expected:

```
debug* up infinite 3 idle compute[01-03]
```

12.47 IMPORTANT Node States

State	Meaning
idle	available
alloc	allocated
mix	partially used
drain	unavailable
down	offline

12.48 Check Output Files

After completion:

```
ls
```

Expected:

```
101.out
101.err
```

View output:

```
cat 101.out
```

Expected:

- serial runtime
- parallel runtime
- speedup
- efficiency

12.49 IMPORTANT HPC Performance Metrics

The program calculates:

Metric	Meaning
Speedup	serial_time / parallel_time
Efficiency	speedup / processes

Ideal values:

Metric	Ideal
Speedup	near process count
Efficiency	near 1.0

Real systems experience:

- communication overhead
- synchronization cost
- MPI latency
- reduction overhead

12.50 Cancel Jobs

Cancel job:

```
scancel <jobid>
```

Example:

```
scancel 101
```

Cancel all user jobs:

```
scancel -u $USER
```

12.51 Interactive Slurm Execution

Launch interactive shell:

```
srun --pty bash
```

Distributed hostname test:

```
srun -N 3 -n 3 hostname
```

Expected:

- hostnames from allocated nodes

This validates:

- scheduler allocation
- distributed execution
- runtime communication

12.52 IMPORTANT HPC Workflow

Typical HPC workflow:

```
Load software
    ↓
Compile application
    ↓
Prepare Slurm script
    ↓
Submit job
    ↓
Monitor execution
    ↓
Analyze outputs
```

12.53 IMPORTANT Production HPC Concept

Production HPC systems usually separate:

- operating system packages
- scientific software stack
- compiler stack
- MPI runtime stack
- scheduler integration

exactly like this setup.
This architecture provides:

- reproducibility
- scalability
- centralized administration
- software consistency
- runtime portability

13 Centralized Identity Management using OpenLDAP + nslcd

Goal:

Transform the cluster from:

```
local-user-only infrastructure
```

into:

```
centralized multi-user HPC environment
```

using:

- OpenLDAP
- nslcd
- NSS
- PAM

This phase introduces centralized user management for the HPC cluster. After completion:

- users are created once centrally
- login node recognizes LDAP users
- compute nodes recognize LDAP users
- centralized authentication works cluster-wide
- automatic home directory creation works
- Slurm can later use centralized identities

13.1 Important Architecture

Cluster roles:

Node	Role
master	LDAP server
login	LDAP client
compute01	LDAP client
compute02	LDAP client
compute03	LDAP client

Important concept:

```
Only master runs LDAP SERVER (slapd).

Login and compute nodes run LDAP CLIENTS (nslcd).
```

13.2 Important Linux Authentication Concepts

Before starting, understand the layers.

Component	Purpose
LDAP	centralized user database
NSS	user/group lookup
PAM	authentication
nslcd	bridge between Linux and LDAP

Authentication flow:

```
SSH Login
   ↓
PAM
   ↓
nslcd
   ↓
LDAP server
```

13.3 Install OpenLDAP Server on Master Node

Run ONLY on:

```
master
```

Install LDAP packages:

```
sudo dnf install -y \
openldap \
openldap-servers \
openldap-clients
```

Package explanation:

Package	Purpose
openldap	LDAP utilities
openldap-servers	slapd server daemon
openldap-clients	ldapsearch, ldapadd etc

13.4 Enable LDAP Server

Run ONLY on:

```
master
```

Enable and start slapd:

```
sudo systemctl enable --now slapd
```

Verify:

```
systemctl status slapd
```

Expected:

```
active (running)
```

13.5 Generate LDAP Admin Password Hash

Run ONLY on:

```
master
```

Generate secure LDAP admin password hash:

```
slappasswd
```

Enter desired password.
Example:

```
cluster123
```

Example output:

```
{SSHA}xxxxxxxxxxxxxxx
```

Important:

```
Copy this hash carefully.
```

It will be used in LDAP database configuration.

13.6 Configure LDAP Database

Run ONLY on:

```
master
```

Create LDAP database configuration file:

```
vim ~/db.ldif
```

Paste:

```
dn: olcDatabase={2}mdb,cn=config
changetype: modify
replace: olcSuffix
olcSuffix: dc=hpc,dc=local

dn: olcDatabase={2}mdb,cn=config
changetype: modify
replace: olcRootDN
olcRootDN: cn=Manager,dc=hpc,dc=local

dn: olcDatabase={2}mdb,cn=config
changetype: modify
replace: olcRootPW
olcRootPW: PASTE_HASH_HERE
```

Replace:

```
PASTE_HASH_HERE
```

with the hash generated using:

```
slappasswd
```

13.7 Important LDAP Concepts

LDAP uses a hierarchical tree called:

```
DIT -- Directory Information Tree
```

Example structure:

```
dc=hpc,dc=local
  ou=People
  ou=Groups
```

Meaning of terms:

Term	Meaning
dc	domain component
ou	organizational unit
cn	common name

13.8 Apply LDAP Database Configuration

Run ONLY on:

```
master
```

Apply configuration:

```
sudo ldapmodify -Y EXTERNAL -H ldapi:/// \
-f ~/db.ldif
```

Expected:

```
modifying entry
```

13.9 Import LDAP Schemas

Schemas define:

- user attributes
- group attributes
- UNIX account structure

Run ONLY on:

```
master
```

Import cosine schema:

```
sudo ldapadd -Y EXTERNAL -H ldapi:/// \
-f /etc/openldap/schema/cosine.ldif
```

Import nis schema:

```
sudo ldapadd -Y EXTERNAL -H ldapi:/// \
-f /etc/openldap/schema/nis.ldif
```

Import inetorgperson schema:

```
sudo ldapadd -Y EXTERNAL -H ldapi:/// \
-f /etc/openldap/schema/inetorgperson.ldif
```

Schema explanation:

Schema	Purpose
cosine	basic directory attributes
nis	UNIX/Linux account support
inetorgperson	user/person identity support

13.10 Create LDAP Base Tree

Run ONLY on:

```
master
```

Create base LDAP tree:

```
vim ~/base.ldif
```

Paste:

```
dn: dc=hpc,dc=local
objectClass: top
objectClass: dcObject
objectClass: organization
o: HPC Cluster
dc: hpc

dn: ou=People,dc=hpc,dc=local
objectClass: organizationalUnit
ou: People

dn: ou=Groups,dc=hpc,dc=local
objectClass: organizationalUnit
ou: Groups
```

Apply tree:

```
ldapadd -x \
-D "cn=Manager,dc=hpc,dc=local" \
-W \
-f ~/base.ldif
```

Enter LDAP admin password.

13.11 Verify LDAP Tree

Run ONLY on:

```
master
```

Verify tree:

```
ldapsearch -x -b "dc=hpc,dc=local"
```

Expected:

- dc=hpc,dc=local
- ou=People
- ou=Groups

13.12 Create LDAP Group

Run ONLY on:

```
master
```

Create group file:

```
vim ~/group.ldif
```

Paste:

```
dn: cn=hpcusers,ou=Groups,dc=hpc,dc=local
objectClass: posixGroup
cn: hpcusers
gidNumber: 10000
```

Add group:

```
ldapadd -x \
-D "cn=Manager,dc=hpc,dc=local" \
-W \
-f ~/group.ldif
```

13.13 Create LDAP User Password Hash

Run ONLY on:

```
master
```

Generate password hash:

```
slappasswd
```

Example password:

```
test123
```

Copy generated hash carefully.

13.14 Create LDAP User

Run ONLY on:

```
master
```

Create user file:

```
vim ~/user.ldif
```

Paste:

```
dn: uid=testUser,ou=People,dc=hpc,dc=local
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
cn: Test User
sn: User
uid: testUser
uidNumber: 10001
gidNumber: 10000
homeDirectory: /home/testUser
loginShell: /bin/bash
userPassword: PASTE_HASH_HERE
```

Replace:

```
PASTE_HASH_HERE
```

with actual hash from:

```
slappasswd
```

Add user:

```
ldapadd -x \
-D "cn=Manager,dc=hpc,dc=local" \
-W \
-f ~/user.ldif
```

13.15 Important LDAP User Attributes

Attribute	Purpose
uid	username
uidNumber	UNIX UID
gidNumber	primary group
homeDirectory	user home
loginShell	login shell
cn	common name
sn	surname

13.16 Verify LDAP User

Run ONLY on:

```
master
```

Verify user:

```
ldapsearch -x \
-b "dc=hpc,dc=local" uid=testUser
```

Expected:

- LDAP user entry

13.17 Configure LDAP Client on Login Node

Now begin client-side integration.
Run ONLY on:

```
login
```

Install LDAP client packages:

```
sudo dnf install -y \
openldap-clients \
nss-pam-ldapd
```

13.18 Configure nslcd

Run ONLY on:

```
login
```

Create configuration:

```
sudo vim /etc/nslcd.conf
```

Paste:

```
uid nslcd
gid ldap

uri ldap://master
base dc=hpc,dc=local
```

Explanation:

Parameter	Purpose
uri	LDAP server
base	LDAP search root

Secure configuration:

```
sudo chmod 600 /etc/nslcd.conf
```

13.19 Enable nslcd

Run ONLY on:

```
login
```

Enable service:

```
sudo systemctl enable --now nslcd
```

Verify:

```
systemctl status nslcd
```

Expected:

```
active (running)
```

13.20 Verify LDAP Connectivity

Run ONLY on:

```
login
```

Test LDAP communication:

```
ldapsearch -x \
-H ldap://master \
-b "dc=hpc,dc=local" uid=testUser
```

Expected:

- LDAP user entry

This verifies:

- login node can communicate with master LDAP server

13.21 Configure NSS

Run ONLY on:

```
login
```

Edit:

```
sudo vim /etc/nsswitch.conf
```

Modify lines:

```
passwd: files ldap
group: files ldap
shadow: files ldap
```

Important:

```
This tells Linux:
check local files first, then LDAP.
```

13.22 Restart nslcd

Run ONLY on:

```
login
```

Restart service:

```
sudo systemctl restart nslcd
```

13.23 Verify LDAP User Resolution

Run ONLY on:

```
login
```

Test centralized user lookup:

```
getent passwd testUser
```

Expected:

```
testUser:x:10001:10000:Test User:/home/testUser:/bin/bash
```

Important concept:

```
This proves Linux itself recognizes LDAP users.
```

This is one of the most important LDAP verification steps.

1. NOTE

   ```
   Now we must configure: *master* with EXACT same setup (master should be able to re
   ```

13.24 Install Automatic Home Directory Support

Run ONLY on:

```
login
```

Install package:

```
sudo dnf install -y oddjob-mkhomedir
```

Enable service:

```
sudo systemctl enable --now oddjobd
```

Purpose:

```
Automatically creates user home directory
during first successful login.
```

13.25 Configure PAM Authentication

Run ONLY on:

```
login
```

Edit:

```
sudo vim /etc/pam.d/system-auth
```

Add LDAP lines in appropriate sections.
Add in auth section:

```
auth        sufficient    pam_ldap.so use_first_pass
```

Add in account section:

```
account     sufficient    pam_ldap.so
```

Add in password section:

```
password    sufficient    pam_ldap.so use_authtok
```

Add in session section:

```
session     optional      pam_ldap.so
session     optional      pam_mkhomedir.so
```

Important:

```
Do NOT remove pam_unix.so lines.
```

13.26 Configure Second PAM File

Run ONLY on:

```
login
```

Edit:

```
sudo vim /etc/pam.d/password-auth
```

Add SAME LDAP PAM lines.
Reason:

```
Different login methods use different PAM stacks.
```

13.27 Restart Services

Run ONLY on:

```
login
```

Restart services:

```
sudo systemctl restart nslcd
sudo systemctl restart oddjobd
```

13.28 Verify LDAP Authentication

Run ONLY on:

```
login
```

Test login:

```
su - testUser
```

Enter LDAP password.
Expected:

```
Creating directory '/home/testUser'
```

Successful login indicates:

- LDAP authentication working
- PAM integration working
- automatic home directory creation working

13.29 Verify User Home

Run ONLY on:

```
login
```

After login:

```
pwd
```

Expected:

```
/home/testUser
```

1. NOW WE MUST REPLICATE TO COMPUTE NODES
   Currently:

   ```
   login node works fully
   ```

   Now we configure:

   - compute01
   - compute02
   - compute03

   with EXACT same client setup.

13.30 Troubleshooting

1. Important Slurm + LDAP Troubleshooting
   This section explains one of the MOST common HPC setup issues.
2. Symptom
   LDAP users:

   - can SSH login
   - can use su
   - appear in getent passwd

   BUT:

   ```
   sbatch submit.sh
   ```

   results in:

   - jobs stuck in pending
   - BeginTime state
   - scheduler instability

- sinfo hanging
- Slurm credential errors
- slurmctld crashes

3. Root Cause

   Slurm internally works using:

   - UID
   - GID
   - NSS identity resolution

   When launching jobs, Slurm performs internal user identity lookups.

   If master node cannot resolve LDAP users correctly through:

   - NSS
   - nslcd
   - LDAP

   then:

   - Slurm credential generation fails
   - scheduler may crash
   - jobs cannot launch

   Common error:

   ```
   _fill_cred_gids: getpwuid failed for uid=10001
   ```

   Meaning:

   ```
   Slurm cannot resolve LDAP user identity correctly.
   ```

4. Important Verification Commands

   Run on:

   ```
   master
   login
   compute nodes
   ```

Verify LDAP resolution:

```
getent passwd testUser
id testUser
```

Expected:

```
uid=10001(testUser) gid=10000(hpcusers)
```

These commands MUST work correctly on ALL cluster nodes.

5. Important Master Node Verification

   The master node MUST also resolve LDAP users correctly.

   Verify:

   ```
   getent passwd testUser
   ```

   If master cannot resolve LDAP users:

   - slurmctld may fail
   - scheduler may crash
   - jobs remain pending

   Verify nslcd configuration on master:

   ```
   sudo vim /etc/nslcd.conf
   ```

   Example:

   ```
   uid nslcd
   gid ldap

   uri ldap://master
   base dc=hpc,dc=local
   ```

   Verify NSS configuration:

   ```
   sudo vim /etc/nsswitch.conf
   ```

Ensure:

```
passwd: files ldap
group: files ldap
shadow: files ldap
```

Restart LDAP client:

```
sudo systemctl restart nslcd
```

6. Important Slurm Architecture

   Correct architecture:

Node	Service
master	slurmctld
login	login services only
compute01	slurmd
compute02	slurmd
compute03	slurmd

   IMPORTANT:

```
Login node should NOT run slurmd.
```

   Because login node is NOT configured as a compute node in:

```
/etc/slurm/slurm.conf
```

   Attempting to start slurmd on login gives:

```
slurmd: fatal: Unable to determine this slurmd's NodeName
```

   This is NORMAL and expected.

   Correct action on login node:

```
sudo systemctl stop slurmd
sudo systemctl disable slurmd
```

Expected afterward:

```
inactive (dead)
```

This is CORRECT for login nodes.

7. Important Slurm Recovery Commands
   Whenever:

   - LDAP users are added
   - NSS configuration changes
   - PAM configuration changes
   - nslcd configuration changes

   restart Slurm services.
   Restart controller on master:

   ```
   sudo systemctl restart slurmctld
   ```

   Restart compute daemons on compute nodes:

   ```
   sudo systemctl restart slurmd
   ```

   Restart LDAP client:

   ```
   sudo systemctl restart nslcd
   ```

   Verify cluster:

   ```
   sinfo
   ```

   Expected:

   ```
   debug* up infinite 3 idle compute[01-03]
   ```

8. Important Scheduler Debugging
   If:

```
sinfo
```

hangs indefinitely:

Possible cause:

```
slurmctld crashed
```

Verify:

```
systemctl status slurmctld
```

or:

```
journalctl -u slurmctld -n 50
```

13.31 Current Cluster State After LDAP Setup

Cluster now contains:

Component	Status
OpenLDAP server	working
centralized users	working
centralized authentication	working
nslcd integration	working
NSS integration	working
PAM authentication	working
automatic home creation	working

13.32 Important Debugging Commands

Verify LDAP tree:

```
ldapsearch -x -b "dc=hpc,dc=local"
```

Verify LDAP user:

```
ldapsearch -x -b "dc=hpc,dc=local" uid=testUser
```

Verify LDAP authentication:

```
ldapwhoami -x \
-D "uid=testUser,ou=People,dc=hpc,dc=local" \
-W \
-H ldap://master
```

Verify Linux user resolution:

```
getent passwd testUser
```

Verify nslcd service:

```
systemctl status nslcd
```

13.33 Important Architecture Understanding

Master node:

- provides centralized identity database

Login/compute nodes:

- authenticate users
- resolve LDAP users
- create user sessions

This separation is fundamental in distributed authentication systems.

14 Automated LDAP User Creation Script

Managing users manually using individual LDIF files becomes difficult as the number of users increases.

To simplify user management, an automation script can be used to:

- create LDAP group
- create LDAP user
- generate password automatically
- generate password hash automatically
- add entries into LDAP

This script automates complete LDAP user creation.
IMPORTANT:

```
The script creates:

- private group for user
- LDAP user entry
- hashed LDAP password

automatically.
```

14.1 Script Location

Example:

```
cd ~/ldap-scripts
ls
```

Expected:

```
createUser.sh
```

14.2 User Creation Script

Create file:

```
vim createUser.sh
```

Paste the complete script exactly as shown below:

```
#!/bin/bash

LDAP_ADMIN="cn=Manager,dc=hpc,dc=local"
BASE_DN="dc=hpc,dc=local"

USERNAME=$1
UIDNUM=$2
GIDNUM=$3

if [ $# -ne 3 ]; then
    echo "Usage: $0 <username> <uid> <gid>"
    exit 1
```

```
fi

# Check if user already exists
USER_EXISTS=$(ldapsearch -x \
-b "ou=People,${BASE_DN}" "(uid=${USERNAME})" dn \
| grep "^dn:")

if [ ! -z "$USER_EXISTS" ]; then
    echo "ERROR: User ${USERNAME} already exists!"
    exit 1
fi

# Check if group already exists
GROUP_EXISTS=$(ldapsearch -x \
-b "ou=Groups,${BASE_DN}" "(cn=${USERNAME})" dn \
| grep "^dn:")

if [ ! -z "$GROUP_EXISTS" ]; then
    echo "ERROR: Group ${USERNAME} already exists!"
    exit 1
fi

PASSWORD="${USERNAME}@@123"

PASSWORD_HASH=$(slappasswd -s "$PASSWORD")

echo
echo "================================="
echo "Creating LDAP User"
echo "================================="
echo "Username : ${USERNAME}"
echo "UID      : ${UIDNUM}"
echo "GID      : ${GIDNUM}"
echo "================================="

# Create private group LDIF
cat > /tmp/${USERNAME}_group.ldif <<EOF
dn: cn=${USERNAME},ou=Groups,${BASE_DN}
objectClass: posixGroup
cn: ${USERNAME}
```

```
gidNumber: ${GIDNUM}
EOF

# Create user LDIF
cat > /tmp/${USERNAME}.ldif <<EOF
dn: uid=${USERNAME},ou=People,${BASE_DN}
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
cn: ${USERNAME}
sn: ${USERNAME}
uid: ${USERNAME}
uidNumber: ${UIDNUM}
gidNumber: ${GIDNUM}
homeDirectory: /home/${USERNAME}
loginShell: /bin/bash
userPassword: ${PASSWORD_HASH}
EOF

echo
echo "Adding LDAP group..."

ldapadd -x \
-D "${LDAP_ADMIN}" \
-W \
-f /tmp/${USERNAME}_group.ldif

if [ $? -ne 0 ]; then
    echo "ERROR: Failed to create LDAP group!"
    exit 1
fi

echo
echo "Adding LDAP user..."

ldapadd -x \
-D "${LDAP_ADMIN}" \
-W \
-f /tmp/${USERNAME}.ldif
```

```
if [ $? -ne 0 ]; then
    echo "ERROR: Failed to create LDAP user!"
    exit 1
fi

echo
echo "================================="
echo "LDAP User Created Successfully"
echo "================================="
echo "Username : ${USERNAME}"
echo "Password : ${PASSWORD}"
echo "UID      : ${UIDNUM}"
echo "GID      : ${GIDNUM}"
echo "================================="
```

14.3 Make Script Executable

Before using the script:

```
chmod +x createUser.sh
```

14.4 Understanding Script Arguments

The script requires THREE arguments:

Argument	Meaning
username	LDAP username
uid	UNIX UID
gid	UNIX GID

Syntax:

```
./createUser.sh <username> <uid> <gid>
```

Example:

```
./createUser.sh student01 11001 11001
```

Explanation:

Value	Meaning
student01	username
11001	UID
11001	GID

IMPORTANT:

```
The script creates a private group automatically.

Therefore username and group name become identical.
```

Example:

Entity	Created Automatically
User	student01
Group	student01
Home Directory	/home/student01

14.5 Default Password Format

The script automatically creates password using:

```
username@@123
```

Example:

Username	Password
student01	student01@@123
student02	student02@@123

IMPORTANT:

```
Users should change password after first login.
```

14.6 What the Script Automatically Does

The script performs following operations automatically:

Operation	Description
Check existing user	avoids duplicate users
Check existing group	avoids duplicate groups
Generate password hash	secure LDAP password
Create LDAP group LDIF	automatic
Create LDAP user LDIF	automatic
Add LDAP group	automatic
Add LDAP user	automatic

14.7 Example User Creation

Example:

```
./createUser.sh student02 11002 11002
```

Expected output:

```
==================================
Creating LDAP User
==================================
Username : student02
UID      : 11002
GID      : 11002
==================================
```

LDAP admin password will be requested.
After successful creation:

```
==================================
LDAP User Created Successfully
==================================
Username : student02
Password : student02@@123
UID      : 11002
GID      : 11002
==================================
```

14.8 Verify User Creation

Verify user using:

```
getent passwd student02
```

or:

```
id student02
```

Expected:

```
uid=11002(student02)
```

Verify LDAP entry:

```
ldapsearch -x \
-b "dc=hpc,dc=local" uid=student02
```

14.9 Verify Login

Test login:

```
su - student02
```

or:

```
ssh student02@login
```

Expected:

```
successful login
```

14.10 Important UID/GID Notes

IMPORTANT:

```
UID and GID values must remain unique cluster-wide.
```

Avoid:

- duplicate UID
- duplicate GID

because duplicate IDs can create:

- ownership conflicts
- wrong usernames
- incorrect file ownership
- Slurm identity problems

Example of BAD configuration:

User	UID
student01	11001
student02	11001

This can cause:

```
whoami returning wrong username
```

Always use unique UID/GID values.

14.11 Important HPC Identity Concept

In HPC clusters:

- NFS
- Slurm
- MPI
- LDAP
- SSH

all depend on:

```
consistent unique UID/GID mappings
```

across the entire cluster.

This is one of the MOST important concepts in centralized HPC authentication.

15 Automated LDAP User Deletion Script

Managing users manually using:

- ldapdelete
- group deletion
- home directory cleanup

can become repetitive and error-prone.

To simplify cleanup operations, an automated LDAP deletion script can be used.

This script automates:

- LDAP user deletion
- optional LDAP group deletion
- optional home directory deletion

IMPORTANT:

```
The script provides interactive cleanup options
for safer user management.
```

15.1 Script Location

Example:

```
cd ~/ldap-scripts
ls
```

Expected:

```
deleteUser.sh
```

15.2 User Deletion Script

Create file:

```
vim deleteUser.sh
```

Paste the complete script exactly as shown below:

```
#!/bin/bash

LDAP_ADMIN="cn=Manager,dc=hpc,dc=local"
BASE_DN="dc=hpc,dc=local"

USERNAME=$1

if [ $# -ne 1 ]; then
    echo "Usage: $0 <username>"
    exit 1
fi

echo
echo "================================="
echo "Deleting LDAP User"
echo "================================="
echo "Username : ${USERNAME}"
echo "================================="

echo
echo "Deleting LDAP user entry..."
```

```
ldapdelete -x \
-D "${LDAP_ADMIN}" \
-W \
"uid=${USERNAME},ou=People,${BASE_DN}"

if [ $? -ne 0 ]; then
    echo "ERROR: Failed to delete LDAP user!"
    exit 1
fi

echo
echo "LDAP user deleted successfully."

# Ask about deleting group
echo
read -p "Delete LDAP group '${USERNAME}'? (y/n): " DELETE_GROUP

case $DELETE_GROUP in
    y|Y)

        ldapdelete -x \
        -D "${LDAP_ADMIN}" \
        -W \
        "cn=${USERNAME},ou=Groups,${BASE_DN}"

        if [ $? -eq 0 ]; then
            echo "LDAP group deleted successfully."
        else
            echo "WARNING: Failed to delete LDAP group."
        fi
        ;;

    n|N)
        echo "Skipping LDAP group deletion."
        ;;

    *)
        echo "Invalid option. Skipping group deletion."
        ;;
esac
```

```
# Ask about deleting home directory
echo
read -p "Delete home directory '/home/${USERNAME}'? (y/n): " DELETE_HOME

case $DELETE_HOME in
    y|Y)

        if [ -d "/home/${USERNAME}" ]; then
            sudo rm -rf "/home/${USERNAME}"

            if [ $? -eq 0 ]; then
                echo "Home directory deleted successfully."
            else
                echo "WARNING: Failed to delete home directory."
            fi
        else
            echo "Home directory does not exist."
        fi
        ;;

    n|N)
        echo "Skipping home directory deletion."
        ;;

    *)
        echo "Invalid option. Skipping home deletion."
        ;;
esac

echo
echo "================================="
echo "User cleanup completed"
echo "================================="
```

15.3 Make Script Executable

Before using the script:

```
chmod +x deleteUser.sh
```

15.4 Understanding Script Arguments

The script requires ONE argument:

Argument	Meaning
username	LDAP username

Syntax:

```
./deleteUser.sh <username>
```

Example:

```
./deleteUser.sh student02
```

15.5 What the Script Automatically Does

The script performs following operations automatically:

Operation	Description
Delete LDAP user	removes LDAP account
Optional group deletion	removes LDAP group if requested
Optional home deletion	removes user home if requested
Interactive confirmation	prevents accidental cleanup

15.6 Example User Deletion

Example:

```
./deleteUser.sh student02
```

Expected output:

```
==================================
Deleting LDAP User
==================================
Username : student02
==================================
```

LDAP admin password will be requested.
After successful deletion:

```
LDAP user deleted successfully.
```

The script then asks:

```
Delete LDAP group 'student02'? (y/n):
```

If:

```
y
```

is selected, corresponding LDAP group will also be removed.
Next prompt:

```
Delete home directory '/home/student02'? (y/n):
```

If:

```
y
```

is selected, user home directory will also be deleted.

15.7 Important Group Deletion Notes

IMPORTANT:

```
Do NOT delete shared groups accidentally.
```

Example:

```
hpcusers
```

may contain multiple users.
Deleting shared groups can create:

- broken group mappings
- permission problems
- Slurm identity issues

The script therefore asks confirmation before deleting groups.

15.8 Important Home Directory Notes

Deleting home directories permanently removes:

- source codes
- job scripts
- outputs
- datasets
- SSH keys
- configuration files

IMPORTANT:

```
Home directory deletion is irreversible.
```

Always verify before deleting.

15.9 Verify User Removal

Verify deletion:

```
getent passwd student02
```

Expected:

```
no output
```

Verify LDAP entry removal:

```
ldapsearch -x \
-b "dc=hpc,dc=local" uid=student02
```

Expected:

```
no entries returned
```

Verify login failure:

```
su - student02
```

Expected:

```
user does not exist
```

15.10 Important HPC User Management Concept

In centralized HPC environments:

- user lifecycle management
- account cleanup
- UID/GID consistency
- group management

are critical administrative tasks.
Improper deletion can create:

- orphaned files
- broken permissions
- Slurm scheduling issues
- incorrect ownership mappings

Therefore automated and controlled cleanup procedures are highly important in production HPC systems.

16 Automated LDAP Password Change Script

Managing passwords manually using LDIF files becomes inconvenient in centralized LDAP environments.

To simplify password management, an automated password change script can be used.

This script automates:

- LDAP password update
- password hash generation
- secure password input
- LDAP password modification

IMPORTANT:

```
Because authentication is centralized through LDAP,
password changes automatically work cluster-wide.
```

After changing password:

- login node recognizes new password
- compute nodes recognize new password
- master recognizes new password

without changing anything individually on nodes.

16.1 Script Location

Example:

```
cd ~/ldap-scripts
ls
```

Expected:

```
changePassword.sh
```

16.2 Password Change Script

Create file:

```
vim changePassword.sh
```

Paste the complete script exactly as shown below:

```
#!/bin/bash

LDAP_ADMIN="cn=Manager,dc=hpc,dc=local"
BASE_DN="dc=hpc,dc=local"

USERNAME=$1

if [ $# -ne 1 ]; then
    echo "Usage: $0 <username>"
    exit 1
fi
```

```
# Check if user exists
USER_EXISTS=$(ldapsearch -x \
-b "ou=People,${BASE_DN}" "(uid=${USERNAME})" dn \
| grep "^dn:")

if [ -z "$USER_EXISTS" ]; then
    echo "ERROR: User ${USERNAME} does not exist!"
    exit 1
fi

echo
echo "================================="
echo "LDAP Password Change"
echo "================================="
echo "Username : ${USERNAME}"
echo "================================="

# Take password securely
echo
read -s -p "Enter new password: " PASSWORD1
echo

read -s -p "Confirm new password: " PASSWORD2
echo

# Check password match
if [ "$PASSWORD1" != "$PASSWORD2" ]; then
    echo
    echo "ERROR: Passwords do not match!"
    exit 1
fi

# Generate LDAP password hash
PASSWORD_HASH=$(slappasswd -s "$PASSWORD1")

# Create LDIF file
cat > /tmp/${USERNAME}_password.ldif <<EOF
dn: uid=${USERNAME},ou=People,${BASE_DN}
changetype: modify
```

```
replace: userPassword
userPassword: ${PASSWORD_HASH}
EOF

echo
echo "Updating LDAP password..."

ldapmodify -x \
-D "${LDAP_ADMIN}" \
-W \
-f /tmp/${USERNAME}_password.ldif

if [ $? -ne 0 ]; then
    echo
    echo "ERROR: Failed to change password!"
    exit 1
fi

echo
echo "================================="
echo "Password updated successfully"
echo "================================="
echo "Username : ${USERNAME}"
echo "================================="
```

16.3 Make Script Executable

Before using the script:

```
chmod +x changePassword.sh
```

16.4 Understanding Script Arguments

The script requires ONE argument:

Argument	Meaning
username	LDAP username

Syntax:

```
./changePassword.sh <username>
```

Example:

```
./changePassword.sh student01
```

16.5 What the Script Automatically Does

The script performs following operations automatically:

Operation	Description
Verify LDAP user exists	prevents invalid modifications
Secure password input	password hidden during typing
Password confirmation	avoids typing mistakes
Generate password hash	secure LDAP password storage
Modify LDAP password	centralized password update

16.6 Example Password Change

Example:

```
./changePassword.sh student01
```

Expected output:

```
================================
LDAP Password Change
================================
Username : student01
================================
```

The script asks:

```
Enter new password:
```

Then:

```
Confirm new password:
```

LDAP admin password will also be requested.
After successful update:

```
================================
Password updated successfully
================================
Username : student01
================================
```

16.7 Verify Password Change

Test login using updated password:

```
su - student01
```

or:

```
ssh student01@login
```

Expected:

```
successful login using new password
```

Because LDAP authentication is centralized, the updated password works automatically on:

- login node
- compute nodes
- master node

16.8 Important Password Management Notes

IMPORTANT:

```
LDAP stores password hashes, not plain-text passwords.
```

The script automatically generates secure LDAP-compatible password hashes using:

```
slappasswd
```

This improves security by avoiding direct plain-text password storage.